Appendices: Evaluation Details and Code Example

These appendices provide more detail on our evaluation environment and benchmarking. They are intended to reinforce the results presented in Section 5.

Appendix A: Benchmarking and Experimental Setup

We validate GHL in three steps. First, we evaluate the performance of the VF3 implementation. Then, we benchmark GHL against popular general-purpose graph libraries on the SI task. Finally, we apply it to use cases in machine learning and chemistry to illustrate its utility in domain-specific tasks.

A.1 Evaluation of the GHL VF3 Implementation

We first evaluate the implementation of the VF3 algorithm in GHL. This is done in comparison to the BGL's prepackaged VF2 implementation. We utilize the publicly available SI dataset [20], which consists of 1170 target graphs varying in size between 200 to 800 nodes and 598 to 7200 edges, each queried for a specific pattern consisting of 20% to 60% of the vertices of the corresponding target graph. As the entries in the dataset vary widely in complexity, we set a time-out for each solver of 5 minutes.

A.2 Comparison with General Graph Libraries

We next evaluate the performance of GHL's subgraph isomorphism search in comparison to three popular general-purpose graph libraries: iGraph (its Python frontend), Graph-tool, and NetworkX, also using the SI dataset. Benchmarking was conducted through the Python front-end by recording the CPU time consumed by the process. To ensure consistency across all libraries, each run was subject to a timeout of 10800 seconds.

A.3 Domain Use Case: SI/SR on Neural Networks

Thirdly, we apply GHL's SI/SR capabilities to DNN computation graphs. The previously described general graph libraries do not provide native support for SR on matches found in the SI step. Thus, we compare against popular domain-specific frameworks Apache TVM and PyTorch FX. Both Apache TVM and PyTorch FX provide SR functionality on neural network computation graphs. We leverage these to ensure a fair comparison.

As a representative deep learning model, we employed the convolutional neural network ResNet architecture [8] at varying depths, namely ResNet50, ResNet101, and ResNet152, consisting of 177/347/517 vertices, respectively, and perform SI/SR operations on a set of target subgraph patterns within each model. The patterns under consideration reflect common transformation strategies in model optimization. Accelerating such SR processes is crucial, as

these transformations are often applied repeatedly during model optimization and neural architecture search, where slow graph edits can compound into substantial delays in exploration and deployment. The target SI/SR patterns (i) normalization variants, where layers that normalize across batches of inputs are substituted with layers that normalize across groups of channels within one input (B.Norm to G.Norm in Figure 4) [23]; (ii) layer fusion, where convolution and normalization layers are combined into a single operation (Conv/B.Norm Fusion) [11]; and (iii) activation function substitution, where ReLU operations are replaced with Leaky ReLU operations (ReLU to L.ReLU) [17]. Using each library's native rewriting capabilities, we exhaustively searched for all occurrences of these patterns in the computation graph and applied substitutive rewrites of the corresponding subgraphs. We measured and reported the execution time required to identify and transform all instances of each tested pattern.

A.4 Domain Use Case: GHLChem for Molecule Substructure Highlighting and Filtering

Finally, to further reinforce the utility of GHL in cross-domain applications, we develop a rudimentary molecular library, GHLChem, which performs simple molecular substructure filtering and highlighting. At the core, the library extends GHL's `BaseNode`/`PrimitiveEdge` with `Atom`/`Bond` classes. The `Atom` class contains member variables representing atom symbol and charge, while the `Bond` class encodes the type of bond between two atoms. The library also defines a `ChemMap ExtendedGraph` class that overloads the drawing functions to create depictions reminiscent of those produced by domain-specific graph libraries. A Python interface allows molecules to be imported into GHLChem from the popular RDKit library.

To benchmark GHLChem, we create an isomorphism that accepts a graph representing a molecular substructure and matches it in a target molecule by comparing atom and bond type. If a match is found it is tagged in the molecule and highlighted in the depiction. We evaluate the performance of this isomorphism by applying it to 6 substructures over 231 molecules sourced from the RDKit repository. We benchmark only its `discover` function first, then the application of the entire isomorphism pipeline, against the `HasSubstructMatch` of the RDKit library.

A.5 Benchmarking environment

All experiments were conducted on a server equipped with AMD EPYC 7763 @ 3.53GHz processor with 64 cores, running in 64-bit mode, featuring 64 MiB L2 cache, 512 MiB L3 cache and 1.96 TiB of RAM. GHL (together with BGL) was compiled using GCC version 13.2.0 with the compiler flags `-O3 -march=native -flto -funroll-loops -DNDEBUG` enabled. For comparison with general-purpose graph libraries, we used Python-igraph version 0.11.8, Graph-Tool version 2.96, and NetworkX version 3.4.2. For comparison on deep neural network transformations, we leveraged Apache TVM version 0.22.0, and PyTorch 2.7.1 (with the

Table 1: Runtime statistics for BGL VF2 algorithm and the GHL VF3 implementation. The GHL VF3 implementation successfully completes more SI problems at significantly lower latency compared to BGL's prepackaged VF2 implementation.

Statistic	VF2 (BGL)	VF3 (this work)
Total completed runs	474	959
Total aborted runs	696	211
Aborted runs where other finished	485	0
Runs where this faster than other	18	456
Mean runtime (s)	19.4	11.3
Median runtime (s)	1.6	0.1

PyTorch FX toolkit). For comparison in molecular filtering and highlighting, we used RDKit version 2025.09.2. All baseline libraries and frameworks were used with their default subgraph matching and rewriting capabilities. Each experiment was repeated 10 times or until total runtime exceeded 5 minutes, and the reported results correspond to the arithmetic mean in these runs.

Appendix B: Comparison to General and Domain Specific Graph Libraries

The following section highlights the key results of our evaluations of the VF3 implementation, GHL efficiency in respect to state-of-the-art general-purpose libraries, and applicability to domain-specific workloads.

B.1 Evaluation of VF3 Algorithm Implementation

Table 1 provides key VF2/VF3 benchmarking runtime statistics. VF3 completes 2x more SI problems with mean/median latencies 1.7x/16x lower than VF2. In 96% of cases VF3 is faster than VF2, not including tests where it completed while VF2 did not. Most importantly, it finds identical matches as the VF2 algorithm in all cases, and does not fail to finish any problems that were able to be completed by the VF2 algorithm, confirming its correctness.

While evaluating the performance of the VF3 implementation, we increased the timeout to 3 hours to gather more results. Even with this long time-out, the VF2 algorithm failed to complete many problems that were completed by VF3 in less than a tenth of a second. To quantify the underlying reasons for this, we calculated the correlation between problem latency and various 1st-order features of the target and pattern graphs, as presented in Table 2. As can be seen, no single parameter highly correlates with latency for the BGL VF2 algorithm, while the latency of the GHL VF3 implementation is highly correlated with the total number of matches found in the target graph. This is reinforced by Figure 3, which plots the latency of every completed SI problem, sorted in ascending order by each parameter, highest correlation first. The latency of VF2 is highly variable, while the VF3 implementation completes in less than one second for all problems

Table 2: Correlation between algorithm latency and parameters for BGL VF2 algorithm and the GHL VF3 implementation. The VF2 algorithm shows little to no correlation between latency and any one parameter, while the VF3 algorithm shows extremely high correlation with the total number of matches between the target and pattern graph.

Parameter	VF2 (BGL)	VF3 (this work)
Pattern → Target Matches	0.307	0.992
Target Edges	0.275	0.052
Pattern Edges	0.192	0.079
Target Vertices	0.1	0.008
Pattern Vertices	0.044	0.053
Target Density	0.006	0.08
Pattern Density	0.005	0.055

up to those with $\sim$>35k matches. The VF2 algorithm's variability could be attributed to the fact that different algorithms perform better on graphs with different characteristics [12,15]. This may be reinforced by the 18 instances where VF2 outperforms VF3, as well as the results presented in Section B.4. However, we prefer to rule out the possibility that the specific implementation of the BGL VF2 algorithm is implemented inefficiently before making such a claim. A deeper study on this front is outside the scope of the current work and may be followed up in future work.

B.2 Subgraph Isomorphism Matching Performance

Table 3 shows the results of our experiments, grouped by the number of vertices of the target graph. Across the benchmark set, GHL outperforms all other libraries in runtime on average, outperforming iGraph, Graph-tool and NetworkX by 2.3x/100x/207x. The reasons for speed-up vary depending on the compared library. While iGraph provides highly optimized C backends, GHL consistently achieves lower mean execution times across all experiments due to the more performant VF3 algorithm. Graph-tool uses BGL as its backend, with a less performant VF2 algorithm than iGraph. NetworkX, as a Python library, is slower than GHL and iGraph, and faster than Graph-tool for target graphs with up to 800 vertices.

Beyond speed, iGraph, NetworkX, and Graph-tool do not support SR primitives. In practice, users must implement ad-hoc graph reconstruction to perform replacements, which is prone to error and incurs additional overhead. In contrast, GHL offers a built-in SI/SR interface that safely applies rewrites. Overall, these results establish that GHL not only surpasses widely adopted generic graph libraries but also combines this performance advantage with ease of use and extensibility.

Table 3: Comparison on the SI dataset [20]. Time is reported in seconds. All reported values are based on the common subset of target–pattern graph pairs used across all experiments. When averaging speedup across all target-pattern pairs, GHL outperforms iGraph/Graph-tool/NetworkX by 2.3x/100x/207x.

Target Graph Size (Number of Samples)	Library	Min	25%	50%	75%	Max	Mean	GHL Speedup
200 (109)	**GHL**	**6.755E-04**	**1.653E-03**	4.122E-02	7.128E-02	15.048	**1.863E-01**	1x
	iGraph	1.446E-03	1.625E-02	**2.813E-02**	**6.811E-02**	**14.177**	4.207E-01	2.26x
	Graph-tool	2.443E-02	7.377E-02	4.736E-01	24.934	1474.942	93.310	500x
	NetworkX	2.911E-02	9.219E-01	2.205	4.076	134.738	5.532	29.7x
400 (79)	**GHL**	**1.729E-03**	**7.310E-02**	1.874E-01	3.213E-01	**5.213E-01**	**2.032E-01**	1x
	iGraph	3.361E-02	8.024E-02	**1.384E-01**	**2.672E-01**	70.900	1.450	7.1x
	Graph-tool	1.721E-01	5.025E-01	1.491	92.315	1331.213	72.773	358.1x
	NetworkX	5.083E-01	6.500	12.756	28.627	156.602	22.096	108.7x
800 (61)	**GHL**	**5.046E-03**	7.388E-01	8.979E-01	1.415	**2.080**	**1.043**	1x
	iGraph	2.080E-01	**4.116E-01**	**7.471E-01**	**7.826E-1**	19.404	1.240	1.19x
	Graph-tool	1.581	4.419	5.646	57.254	490.184	82.180	78.8x
	NetworkX	5.969	55.121	81.820	205.102	285.803	125.609	120x

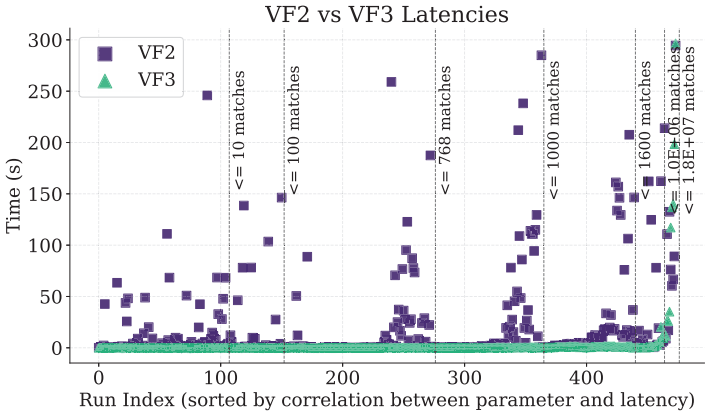

Fig. 3: Latencies of VF2 and VF3 algorithm on the SI dataset [20]. The x-axis represents the index of each run, where runs are ordered according to the correlation between the VF2 latency and the parameter, as illustrated in Table 2. VF2 shows a high degree of variance over the problem set, with low correlation between any one parameter. The VF3 implementation, on the other hand, is highly correlated with the total number of matches found in the target graph.

B.3 Domain A: Subgraph Replacement in Neural Network Graphs

Figure 4 shows the results of our experiments on different ResNet architectures. GHL successfully identified and replaced the target subgraphs in all deep learning models, achieving performance that consistently surpassed domain-specific frameworks. This improvement is attributable to its efficient subgraph matching implementation and its lightweight SR mechanism. Furthermore, these results demonstrate that GHL's domain-agnostic design does not come at the cost of efficiency, enabling optimizations that are competitive with, and in many cases superior to, domain-specific systems.

Beyond performance, GHL enables expressiveness in SR operations beyond the simple pattern definitions supported by PyTorch FX and Apache TVM. Specifically, GHL's `specialize_isomorphism` and `is_isomorphism_valid` hooks allow pattern extensions and validations with arbitrarily complex logic. This capability is particularly valuable in domains such as embedded systems and neural architecture search, where patterns may depend on hardware-specific constraints, search-time heuristics, or adaptive constraints. These features enable GHL to handle tasks such as identifying complex or partially defined workloads and replacing them with hardware-optimized high-performance kernels.

B.4 Domain B: Molecular Substructure Highlighting/Filtering

Figure 5 illustrates the results of GHLChem over the 6 benchmarked substructures for the VF3 and VF2 SI implementations. The results demonstrate an

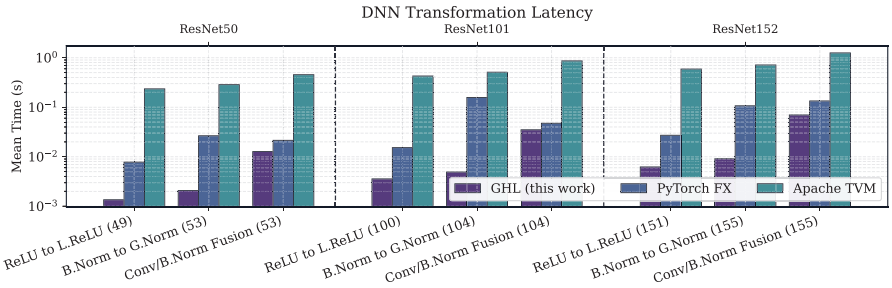

Fig. 4: Comparison of the runtime for three transformations on three ResNet depths. Time in seconds, y axis on logarithmic scale. On average, GHL outperforms PyTorch FX/Apache TVM by 8x/87x.

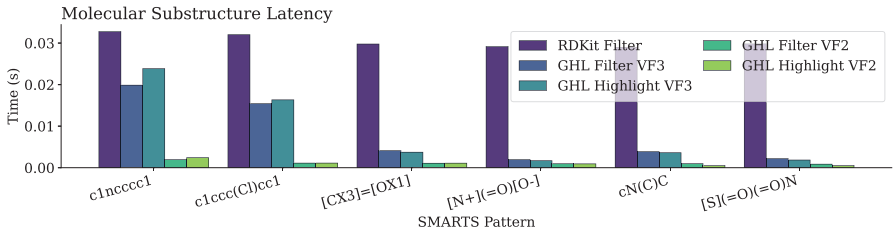

Fig. 5: Latency results for RDKit, VF2, and VF3 molecular filtering and highlighting. GHL demonstrates competitive performance relative to RDKit.

interesting property in contrast to the above evaluations, as the VF2 implementation significantly outperforms VF3 across all tests. This may be attributed to the fact that the graphs under study are significantly smaller and less densely connected in comparison to those of the SI benchmarks and the ResNet DNNs, consisting of only on average 22.8 atoms. VF3 features that make it effective on large, densely connected graphs, such as its complex look-ahead rules, may reduce its effectiveness on small, sparse graphs [15]. Fortunately, GHL makes choosing between the algorithms simple, and in both cases GHLChem finds identical matches to RDKit with competitive performance. The intent here is not to convince users of RDKit to switch to GHL; rather, it is to demonstrate the adaptability of GHL to domain-specific scenarios and encourage researchers in fields without tailored graph libraries to explore how graph theory and algorithms can be quickly and efficiently applied via GHL.

Figure 6 illustrates an example output of the `write_graph` function when applied to a `ChemMap` class that inherits from the `ExtendedGraph` GHL class. The `SubStruct` isomorphism matches the pattern substructure and tags it by modifying the graph in-place for highlighting by the user-defined vertex and edge writer functions. These overridden functions also capture and render domain-

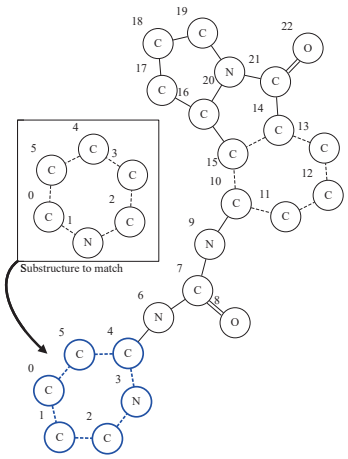

Fig. 6: Example output of GHL's `write_graph` function after substructure highlighting, with overridden rendering functions adapting the output to the chemistry domain.

specific properties such as atom and bond type, demonstrating how GHL can be adapted to the chemistry domain.

Appendix C: Code Listings

The code listings 1 and 2 represent a fully contained example of how GHL may be extended by a user to accomplish a domain-specific task. In the given example, the `BaseNode` is extended to represent a city with a population, with the `Isomorphism` identifying low-population cities and modifying their population value. The primary purpose is to demonstrate the ease of extending GHL for specific tasks, transparent recompilation via `cppimport`, and the utility of the `pybind11` API for calling GHL from Python scripts.

```cpp
// cppimport
/*
<%
# Load the GHL default cppimport config
from graph_hook_library import cppimport
cfg.update(cppimport.default_cfg())
%>
*/
#include <ghl/ghl.h>
#include <ghl/ghl_isomorphisms.h>
#include <pybind11/pybind11.h>
class City : public ghl::BaseNode { // Extend BaseNode to represent a City
public:
  explicit City(int id, int population) : BaseNode(id), population_(population) {}
  int population_ = 100; // City population
};
using ImplementedIsomorphism =
    ghl::UndirectedIsomorphism<std::shared_ptr<ghl::BaseNode>, std::shared_ptr<ghl::PrimitiveEdge>>;

// Declare an Isomorphism that will filter cities under a certain population and modify them
class LanesIso : public ImplementedIsomorphism {
  using ImplementedIsomorphism::ImplementedIsomorphism;
  // Override vertex_comp_function hook to match cities with a population of <= 100
  VertexCompFunction vertex_comp_function() final {
    return [](const GraphType &, const GraphType &target_graph, const VertexDescriptor,
              const VertexDescriptor target_edge) {
      return std::dynamic_pointer_cast<City>(target_graph[target_edge])->population_ <= 100;
    };
  }
  // Modify matched cities by multiplying their population
  InPlaceUpdateFunction inplace_update_function() {
    return [](GraphType &graph, const GraphType &iso_graph, const IsoMap &isomorphism) {
      std::dynamic_pointer_cast<City>(graph[isomorphism.at(0)[0]])->population_ *= 100;
    };
  }
};
// Expose the extended class and isomorphism to Python
PYBIND11_MODULE(listing, m) {
  pybind11::module_::import("graph_hook_library.ghl_bindings");
```

```
  pybind11::class_<City, ghl::BaseNode, std::shared_ptr<City>>(m, "City")
      .def(pybind11::init<int, int>(), pybind11::arg("id"), pybind11::arg("population"))
      .def_readwrite("population", &City::population_);
  pybind11::class_<LanesIso, ImplementedIsomorphism, std::shared_ptr<LanesIso>>(m, "LanesIso")
      .def(pybind11::init<const ImplementedIsomorphism::GraphType &>(), pybind11::arg("pattern"));
}
```

Listing 1: C++ extension of GHL BaseNode and Isomorphism that filters and modifies low-population cities.

```
import cppimport.import_hook  # Import cppimport
import listing # listing.cpp will be transparently recompiled here if previously modified
from graph_hook_library.ghl_bindings import UndirectedGraph, UndirectedExtendedGraph, PrimitiveEdge, apply_isomorphism
target_graph = UndirectedExtendedGraph() # Create graph on which isomorphism will be run
v0 = target_graph.add_vertex(listing.City(0, 100)) # Add city
v1 = target_graph.add_vertex(listing.City(1, 100)) # Add city
e = target_graph.add_edge(v0, v1, PrimitiveEdge(0)) # Add edge
target_graph[v0].population = 500 # Modify city population
pattern_graph = UndirectedExtendedGraph() # Create pattern graph
pattern_graph.add_vertex(listing.City(0, 100)) # Add city
lanes_iso = listing.LanesIso(pattern_graph.graph()) # Create isomorphism with pattern graph
print("City populations:", ", ".join(str(target_graph[v].population) for v in target_graph.vertices()))
# "City populations: 500, 100"
apply_isomorphism(target_graph, lanes_iso, True, True) # Apply isomorphism to graph
print("City populations:", ", ".join(str(target_graph[v].population) for v in target_graph.vertices()))
# "City populations: 500, 10000"
```

Listing 2: Python script that utilizes the above GHL extension with transparent recompilation.